

PROJECT AND TEAM INFORMATION

Project Title :

NEXT MOVE: Warehouse Route Optimizer

Student / Team Information

<i>Team Name:</i> <i>Team #:</i>	
Team member 1 (Team Lead) <i>Last Name, name: student ID: email, picture:</i>	Kamboj, Parv 2510014453 2510014453@geu.ac.in 
Team member 2 <i>Last Name, name: student ID: email, picture:</i>	Chaturvedi, Shivam 25103140054 25103140054@geu.ac.in 
Team member 3 <i>Last Name, name: student ID: email, picture:</i>	Tiwari, Siddhant 2510380013 2510380013@geu.ac.in 

PROPOSAL DESCRIPTION (10 pts)

Motivation

Warehouses keep products in a hierarchy of zones, aisles, racks, shelves and bins. For a single customer order, a picker may have to reach several storage points. When the visit order is not planned, the worker can spend extra time walking between locations. Stock mismatch is another issue: a system may mark an item as available even though the physical slot is empty. This can lead to substitutions, dispatch delays and a lower order value when the missing item was needed to satisfy a delivery condition. NEXT MOVE brings stock checking, order handling, product-location mapping and route planning together in one academic prototype. It identifies the required storage point and can refresh the route when the warehouse situation changes.

State of the Art / Current solution

Existing warehouse software generally supports inventory records, order processing, product locations and picking activities. Graph models and shortest-path methods can be used to reduce unnecessary movement, while handheld devices may provide workers with pick lists and instructions. Large commercial systems can additionally rely on scanners, sensors, databases and dedicated infrastructure. NEXT MOVE is intentionally narrower: it is an explainable academic prototype in which core Data Structures and Algorithms are written in C and the application-level warehouse model is organized with C++.

Project Goals and Milestones

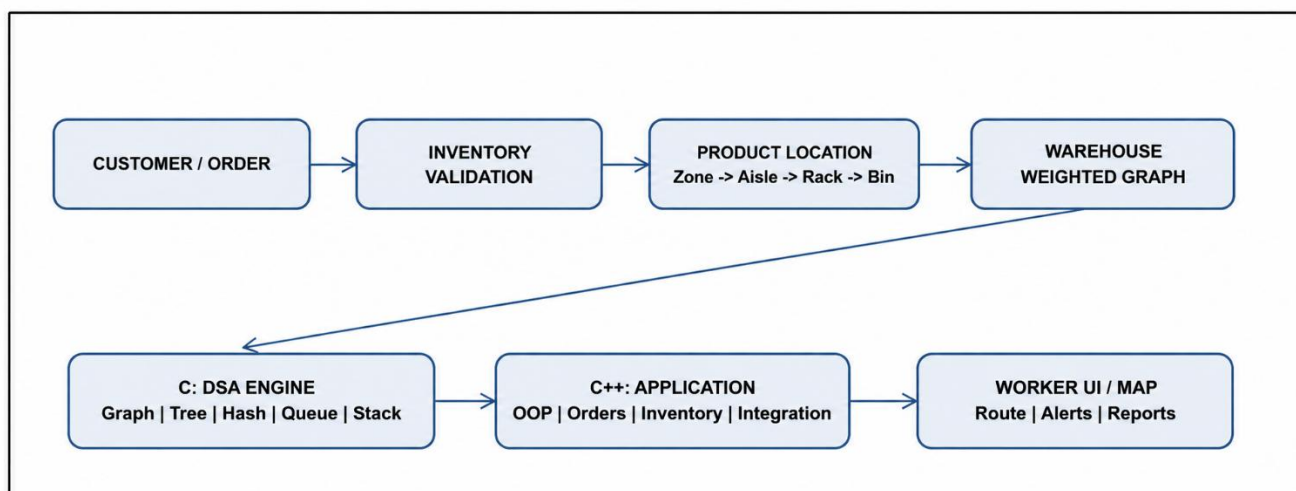
The main objective is to build a working prototype that links order processing, stock verification and warehouse route planning. The planned stages are:

- Create a structured representation of warehouse, product and order information.
- Build and test the required Data Structures in C.
- Add product, inventory and order-management modules.
- Model the warehouse as a weighted graph and demonstrate BFS, DFS and Dijkstra.
- Generate a practical picking sequence for multi-item orders and recompute the route when a path becomes unavailable.
- Connect the C modules with C++ classes and overall application flow.
- Provide a simple worker interface with map, route, alerts and report information.
- Compare regular picking with the optimized route using estimated distance and time.

Project Approach

The proposed system is divided into two connected layers. The C layer will contain the fundamental Data Structures and algorithmic routines, including arrays, linked lists, stacks, queues, priority queues, trees, hashing and graphs, together with searching, sorting, BFS, DFS, Dijkstra and path reconstruction. The C++ layer will represent Product, Order, Warehouse, Zone, Aisle, Rack, Shelf, Bin, Inventory and Worker as classes and will manage the application workflow. The warehouse layout will be stored as a configurable weighted graph, while item locations will follow Zone -> Aisle/Row -> Rack -> Shelf -> Bin. Hashing will support fast lookup, trees will organize categories, queues will manage regular orders and priority queues will move urgent orders ahead. For a multi-product order, the system will create a picking sequence and calculate weighted shortest paths between successive stops. When a route is blocked, the graph state is changed and the remaining path is recalculated.

SYSTEM ARCHITECTURE (HIGH LEVEL DIAGRAM)



Architecture note: The C side performs the core Data Structures and algorithmic work. C++ represents the warehouse entities and controls the application sequence. The worker view presents orders, stock information, map, route guidance, alerts and reports. When an aisle or path is marked unavailable, the graph is updated and a new route is produced for the unfinished portion of the order.

PROJECT OUTCOME / DELIVERABLES

The final deliverable will be a functional NEXT MOVE prototype that demonstrates a complete warehouse-picking workflow. It will cover product records, stock confirmation, rule-based substitute suggestions, order and priority handling, exact item-location mapping, a configurable warehouse graph, graph traversal, Dijkstra-based shortest-path calculation, picking-sequence generation and route updates when conditions change. The worker-facing interface will display the order, warehouse map, route instructions, alerts and basic reports. The prototype will also compare normal and optimized movement using estimated travel distance and time, showing how Data Structures and Algorithms implemented in C can work with C++ object-oriented design in a practical logistics application.

ASSUMPTIONS

The warehouse layout and item positions will be supplied as structured input. Each item can be referenced by Zone, Aisle/Row, Rack, Shelf and Bin. Worker movement may be represented by a simulated position or a fixed checkpoint; indoor GPS is outside the prototype scope. Substitute recommendations will be based on predefined rules rather than a machine-learning model. Route calculation will use non-negative edge costs that can be adjusted for the chosen warehouse layout.

REFERENCES

- T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, MIT Press.
- E. Horowitz, S. Sahni and S. Anderson-Freed, *Fundamentals of Data Structures*.
- B. Stroustrup, *The C++ Programming Language*, Addison-Wesley.
- E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs", *Numerische Mathematik*, 1959.
- Course and laboratory resources for Data Structures and Programming in C++.